Trabajo Práctico Obligatorio

Diseño y Desarrollo de una Aplicación Móvil

Preentrega de diseño + Entrega final de implementación

(v.1.0)

Universidad Argentina de la Empresa
Facultad de Ingeniería y Cs. Exactas
Departamento de Tecnología Informática

El Trabajo Práctico Obligatorio propone diseñar y desarrollar una aplicación Android que responda a un **problema real, concreto y plausible de ser abordado mediante una solución móvil**. La actividad se organiza en dos etapas: una primera instancia de análisis y diseño, que debe ser aprobada antes de comenzar la implementación, y una segunda instancia de desarrollo, validación y defensa del producto.

Índice

1. Propósito del trabajo **3**

2. Modalidad general **3**

3. Principio rector **3**

4. Etapa 1 — Preentrega de análisis y diseño **5**
4.1. Definición del problema . . . 5
4.2. Usuario y contexto de uso . . . 5
4.3. Propuesta de solución . . . 6
4.4. Caso de negocio y propuesta de valor . . . 6
4.5. Alcance y fuera de alcance . . . 7
4.6. Requisitos funcionales . . . 7
4.7. Requisitos no funcionales . . . 8
4.8. Diseño de experiencia — UI, UX y CX . . . 8
4.9. Flujo de pantallas . . . 9
4.10. Tecnologías previstas . . . 9
4.11. Arquitectura propuesta . . . 10
4.12. Persistencia y estrategia de datos . . . 11
4.13. Estrategia Offline First . . . 11
4.14. Repositorio y forma de trabajo . . . 11
4.15. Roles y responsabilidades . . . 12

5. Entregables de la preentrega **12**

6. Defensa de la preentrega **13**

7. Etapa 2 — Implementación y entrega final **15**
7.1. Implementación mínima esperada . . . 15
7.2. Calidad funcional . . . 15
7.3. Calidad de experiencia . . . 16

7.4. Entrega técnica 16
7.5. Demo y defensa final 16

8. Uso de inteligencia artificial **17**

9. Criterios generales de evaluación **17**

10. Cierre **18**

1. Propósito del trabajo

Objetivos

- Identificar un problema real y formularlo desde la necesidad del usuario, no desde la tecnología.
- Diseñar una solución móvil coherente con el contexto de uso.
- Integrar criterios de UI, UX y CX.
- Diseñar una arquitectura basada en MVVM y principios de Clean Architecture.
- Tomar decisiones justificadas sobre persistencia, conectividad, APIs y capacidades del dispositivo.
- Aplicar una estrategia Offline First cuando el problema lo requiera.
- Implementar una aplicación funcional con Kotlin, Android y Jetpack Compose o React Native.
- Documentar y defender las decisiones de diseño y desarrollo.

El objetivo del trabajo no es producir la mayor cantidad posible de pantallas, funcionalidades o tecnologías, sino recorrer un proceso completo de desarrollo móvil: **comprender un problema, diseñar una solución acotada, construirla, probarla y justificar las decisiones tomadas**.

2. Modalidad general

El trabajo se realizará en equipos y tendrá dos instancias obligatorias:

Etapa 1 — Preentrega de análisis y diseño

El equipo deberá definir el problema, usuario, propuesta de valor, alcance, requisitos, experiencia, arquitectura y estrategia tecnológica. Esta instancia debe ser aprobada antes de comenzar formalmente la etapa de implementación.

Etapa 2 — Entrega final de implementación

Una vez aprobada la propuesta, el equipo desarrollará una versión funcional de la aplicación y realizará una demostración y defensa técnica.

Atención

La aprobación de la preentrega no implica que todas las decisiones queden “congeladas”. Durante el desarrollo podrán surgir cambios. Sin embargo, toda modificación significativa de alcance, arquitectura, experiencia o tecnología deberá poder ser explicada y justificada por el equipo.

3. Principio rector

La aplicación deberá surgir de un **problema** y no de una tecnología.

No constituye un problema formular:

> “Queremos hacer una aplicación que use GPS.”

En cambio, una formulación adecuada sería:

> "Las personas que utilizan determinados espacios recreativos desconocen cuáles poseen servicios disponibles, accesibilidad o condiciones adecuadas para ciertas actividades."

A partir de esa situación puede analizarse si ubicación, conectividad, persistencia local, cámara, sensores u otras capacidades móviles aportan valor.

Atención

No se evaluará positivamente agregar tecnologías solamente para "cumplir". El equipo deberá ser capaz de explicar qué problema resuelve cada decisión técnica.

4. Etapa 1 — Preentrega de análisis y diseño

4.1. Definición del problema

El equipo deberá identificar una situación problemática real, concreta y suficientemente acotada como para ser trabajada durante la cursada.

La descripción deberá responder:

- ¿Qué problema existe?
- ¿Quién experimenta ese problema?
- ¿En qué contexto ocurre?
- ¿Cómo se resuelve actualmente?
- ¿Qué dificultades presenta la solución actual?
- ¿Por qué una aplicación móvil podría mejorar esa situación?

El problema deberá redactarse en términos del usuario y su necesidad.

4.2. Usuario y contexto de uso

Se deberá identificar al menos un perfil de usuario principal.

Para dicho usuario deberán describirse:

- características relevantes;
- necesidad principal;
- contexto de uso;
- frecuencia estimada de uso;
- posibles restricciones;
- condiciones de conectividad;
- condiciones ambientales o situacionales relevantes;
- dificultades actuales.

El análisis deberá considerar que una aplicación móvil puede utilizarse en movimiento, durante períodos breves, con conectividad intermitente, con una sola mano o mientras el usuario realiza otra actividad.

Mirada de computación ubicua

Se espera que el diseño recupere la mirada de **Mark Weiser**: la tecnología debería integrarse de manera natural en la actividad del usuario y reducir, cuando sea posible, la carga cognitiva necesaria para utilizarla. La solución debe acompañar el contexto y no obligar al usuario a adaptarse innecesariamente a la tecnología.

4.3. Propuesta de solución

El equipo deberá describir brevemente la aplicación propuesta.

La definición deberá incluir:

- nombre provisorio;
- descripción breve de la solución;
- usuario principal;
- propuesta de valor;
- escenario principal de uso;
- beneficio esperado.

Además, deberá responderse explícitamente:

> **¿Por qué esta solución tiene sentido como aplicación móvil y no simplemente como sitio web o sistema de escritorio?**

Podrán considerarse, cuando aporten valor:

- ubicación;
- cámara;
- sensores;
- almacenamiento local;
- notificaciones;
- conectividad;
- funcionamiento offline;
- interacción contextual;
- integración con otras aplicaciones del dispositivo.

4.4. Caso de negocio y propuesta de valor

El equipo deberá explicar qué valor genera la aplicación.

No es obligatorio que se trate de un proyecto comercial. El valor puede ser:

- económico;
- operativo;
- educativo;
- social;
- ambiental;
- organizacional;

- de accesibilidad;
- de experiencia.

La propuesta deberá poder resumirse mediante la siguiente relación:

Problema → Usuario → Solución → Valor generado

4.5. Alcance y fuera de alcance

El equipo deberá definir qué incluirá la primera versión del producto y qué quedará explícitamente fuera de alcance.

Se espera una solución **acotada y desarrollable**, priorizando pocos flujos completos antes que una gran cantidad de funcionalidades incompletas.

Atención

Una solución pequeña, coherente y completamente funcional tendrá mayor valoración que una propuesta excesivamente ambiciosa que no logre completar sus casos de uso principales.

4.6. Requisitos funcionales

La preentrega deberá incluir entre **3 y 4 requisitos funcionales principales**.

Cada requisito deberá incluir:

- identificador;
- descripción;
- usuario involucrado;
- criterio de aceptación.

Ejemplo:

RF01 — Registrar una observación

Descripción: el usuario podrá registrar una observación seleccionando una categoría, ingresando una descripción y confirmando su ubicación.
Criterio de aceptación: al confirmar la operación, la observación deberá quedar disponible en la aplicación aun cuando el dispositivo pierda posteriormente la conexión.

Otros ejemplos posibles:

- consultar información previamente descargada sin conexión
- registrar información asociada a la ubicación actual
- consultar el detalle de un elemento
- filtrar información según criterios definidos.

4.7. Requisitos no funcionales

Se deberán definir requisitos no funcionales relevantes para el proyecto.

Podrán contemplarse:

- usabilidad
- accesibilidad
- rendimiento
- seguridad
- privacidad
- mantenibilidad
- confiabilidad
- compatibilidad
- comportamiento offline
- recuperación ante errores
- consumo de batería
- uso responsable de sensores
- protección de datos del usuario

Los requisitos deberán formularse de manera observable o verificable.

> **Evitar:** "La aplicación será rápida y fácil de usar."
>
> **Preferir:** "Las operaciones principales de consulta deberán poder realizarse sin conexión cuando exista información previamente sincronizada."

4.8. Diseño de experiencia — UI, UX y CX

El equipo deberá realizar un diseño inicial de la experiencia en **Figma**.

La entrega deberá incluir:

- principales pantallas
- navegación
- componentes principales
- jerarquía visual
- acciones disponibles
- estados relevantes.

No se espera una colección de pantallas aisladas. Debe poder comprenderse **cómo una persona recorre la aplicación para alcanzar un objetivo**.

Cuando corresponda, deberán representarse distintos estados de una misma pantalla:

Carga → contenido → vacío → error → offline

Se tendrá en cuenta:

- claridad de navegación
- consistencia
- feedback del sistema
- prevención y recuperación de errores
- accesibilidad
- carga cognitiva
- contexto real de utilización.

4.9. Flujo de pantallas

El equipo deberá presentar un diagrama que permita comprender el flujo principal de interacción.

Como mínimo deberá incluir:

- punto de entrada
- pantallas principales
- decisiones
- navegación
- operaciones críticas
- posibles finales del recorrido.

Como mínimo deberá documentarse el flujo correspondiente al caso de uso principal.

4.10. Tecnologías previstas

La aplicación deberá desarrollarse utilizando:

- Kotlin
- Android
- React Native
- Jetpack Compose.

Según las necesidades de cada proyecto podrán utilizarse:

- Navigation Compose
- ViewModel
- Coroutines
- Flow / StateFlow

- Room
- Retrofit
- DataStore
- APIs externas
- sensores del dispositivo
- servicios de ubicación
- otras APIs del ecosistema Android.

Cada tecnología relevante deberá incluir una breve justificación.

4.11. Arquitectura propuesta

La aplicación deberá diseñarse aplicando los principios trabajados durante la materia:

- separación de responsabilidades
- MVVM
- principios de Clean Architecture
- gestión explícita del estado
- abstracción de fuentes de datos
- mantenibilidad
- testabilidad.

La preentrega deberá incluir un **diagrama de arquitectura tanto a nivel de Flujo de Datos como de topología de infraestructura**.

Como mínimo deberán distinguirse las siguientes responsabilidades:

Capa	Responsabilidades esperadas
Presentación	UI con Jetpack Compose, estado de pantalla y ViewModel.
Dominio	Modelos de dominio, reglas del negocio, casos de uso cuando correspondan e interfaces.
Datos	Repositorios, fuentes locales, fuentes remotas y mappers cuando sean necesarios.

Un esquema conceptual posible es:

UI → ViewModel → Use Case → Repository → Local / Remote Data Source

Atención

El diagrama no debe ser decorativo. El equipo deberá poder explicar por qué cada responsabilidad se encuentra en determinada capa y cómo circulan los datos.

4.12. Persistencia y estrategia de datos

El equipo deberá identificar qué información requiere persistencia.

Cuando corresponda deberá indicarse:

- entidades principales
- relaciones
- información proveniente de APIs
- información generada por el usuario
- información que deberá almacenarse localmente.

Cuando exista una necesidad real de persistencia estructurada, se espera utilizar **Room**.

4.13. Estrategia Offline First

La propuesta deberá analizar explícitamente qué ocurrirá cuando el dispositivo:

- Tenga conexión
- Pierda conexión
- Recupere conexión
- Disponga de información local desactualizada
- No posea todavía información local.

Cuando el problema lo requiera, deberá plantearse una estrategia **Offline First**.

La preentrega deberá explicar:

- Qué información se almacenará localmente
- Cuál será la fuente remota
- Cuándo se actualizarán los datos
- Qué verá el usuario sin conexión
- Cómo se manejarán errores o conflictos de sincronización.

4.14. Repositorio y forma de trabajo

Antes de realizar la preentrega deberá existir un repositorio del proyecto.

El equipo deberá declarar:

- URL del repositorio
- Integrantes
- Estrategia básica de trabajo
- Ramas utilizadas, si correspondiera

- criterio para commits y pull requests.

Se observará la evolución real del trabajo a través del repositorio.

Atención

La concentración artificial de todo el desarrollo en uno o pocos commits previos a la entrega podrá considerarse evidencia de una metodología de trabajo deficiente.

4.15. Roles y responsabilidades

Cada integrante deberá tener responsabilidades identificables.

La preentrega deberá indicar:

- Nombre del integrante.
- Rol principal.
- Responsabilidades.
- Áreas del proyecto en las que participará.
- Cambios en los miembros del equipo si los hubiere.

Ejemplos de responsabilidades:

- Arquitectura
- UI
- UX
- Persistencia
- Sensores
- Testing
- Documentación
- Integración.

Los roles no implican exclusividad. Todos los integrantes deberán comprender el funcionamiento general de la aplicación.

5. Entregables de la preentrega

La preentrega deberá contener como mínimo:

1. Nombre provisorio de la aplicación.
2. Descripción del problema.
3. Usuario o usuarios principales.
4. Contexto de uso.

5. Propuesta de solución.
6. Justificación de por qué la solución debe ser móvil.
7. Caso de negocio o propuesta de valor.
8. Alcance y fuera de alcance.
9. Entre 3 y 4 requisitos funcionales.
10. Requisitos no funcionales.
11. Diseño en Figma.
12. Flujo de pantallas.
13. Diagrama de arquitectura.
14. Estrategia Offline First, cuando corresponda.
15. Tecnologías previstas y justificación.
16. Repositorio.
17. Integrantes, roles y responsabilidades.

Criterio de aprobación

La preentrega se considerará aprobada cuando exista coherencia suficiente entre:
Problema → Usuario → Experiencia → Funcionalidades → Arquitectura → Tecnología
Una propuesta podrá requerir correcciones si el problema es demasiado amplio, el alcance es excesivo, la experiencia no responde al contexto de uso, los requisitos no son verificables o la arquitectura y las tecnologías no se encuentran suficientemente justificadas.

6. Defensa de la preentrega

Cada equipo podrá ser convocado a una breve presentación de su propuesta.

Todos los integrantes podrán ser consultados.

El equipo deberá ser capaz de explicar:

- Qué problema intenta resolver
- Quién tiene ese problema
- Por qué la solución propuesta tiene sentido
- Por qué se eligió una aplicación móvil
- Cuáles son los requisitos principales
- Cómo será la experiencia de uso
- Cómo se organizarán los datos
- Cómo funcionará la arquitectura

- Qué ocurrirá sin conectividad
- Por qué se eligieron determinadas tecnologías.

La defensa no deberá limitarse a leer la documentación entregada.

7. Etapa 2 — Implementación y entrega final

Una vez aprobada la preentrega, el equipo podrá comenzar formalmente la etapa de desarrollo.

7.1. Implementación mínima esperada

La versión final deberá implementar correctamente los requisitos funcionales comprometidos.

No se considerará suficiente una aplicación que únicamente permita recorrer pantallas. Las operaciones deberán producir efectos reales y consistentes.

La aplicación deberá demostrar integración de los principales conceptos trabajados durante la materia:

- Kotlin
- Android
- Jetpack Compose
- navegación
- estado
- ViewModel
- MVVM
- principios de Clean Architecture
- repositorios
- persistencia
- Room cuando corresponda
- Retrofit cuando exista comunicación remota
- manejo de errores
- funcionamiento offline cuando corresponda
- sensores o capacidades del dispositivo cuando aporten valor.

7.2. Calidad funcional

Se verificará especialmente:

- Funcionamiento de los casos de uso principales
- Manejo de errores
- Navegación consistente
- Actualización correcta del estado
- Persistencia
- Comportamiento con conectividad limitada

- Consistencia entre UI y datos.

7.3. Calidad de experiencia

La versión final deberá contemplar, cuando corresponda:

- Estado de carga;
- Contenido disponible;
- Contenido vacío;
- Error;
- Ausencia de conectividad;
- Permisos rechazados;
- Recuperación ante errores.

La aplicación deberá comunicar claramente qué ocurre y qué acciones puede realizar el usuario.

7.4. Entrega técnica

La entrega final deberá incluir:

- Repositorio completo
- README
- Instrucciones de ejecución
- Arquitectura final
- Tecnologías utilizadas
- Funcionalidades implementadas
- Requisitos funcionales alcanzados
- Decisiones relevantes
- Limitaciones conocidas.

El README deberá permitir que una persona externa al equipo comprenda qué hace la aplicación, cómo está organizada y cómo ejecutarla.

7.5. Demo y defensa final

El equipo deberá realizar una demostración funcional de la aplicación.

Se recomienda estructurarla de la siguiente manera:

Problema → Escenario → Acción del usuario → Respuesta de la aplicación → Valor generado

Durante la defensa podrán realizarse preguntas individuales sobre:

- código
- arquitectura
- estado
- navegación
- persistencia
- networking
- sincronización
- sensores
- decisiones de UX
- organización del trabajo.

Todos los integrantes deberán poder explicar el funcionamiento general de la aplicación.

8. Uso de inteligencia artificial

Uso de IA

Se permite utilizar asistentes de IA durante el desarrollo del TPO como herramienta de consulta, revisión, diagnóstico y apoyo.
El equipo deberá poder comprender, justificar y modificar el código que integra al proyecto.
Durante la defensa podrá solicitarse explicar fragmentos generados o asistidos por IA, identificar sus responsabilidades y justificar por qué fueron incorporados.
Una respuesta generada por IA no constituye por sí misma una justificación técnica.

Antes de incorporar una solución propuesta por IA, el equipo debería poder responder:

- ¿Qué problema intenta resolver este código?
- ¿Qué capa o responsabilidad modifica?
- ¿Qué dependencias agrega?
- ¿Existe una alternativa más simple?
- ¿Cómo comprobamos que funciona?
- ¿Qué ocurriría si falla?

9. Criterios generales de evaluación

La evaluación considerará tanto el **producto** como el **proceso de desarrollo**.

Dimensión	Qué se observará
Problema y usuario	Claridad, pertinencia, contexto y comprensión de la necesidad.
Diseño de solución	Coherencia entre problema, valor, funcionalidades y experiencia.
Alcance	Capacidad de priorizar y definir un producto realizable.
UI / UX / CX	Claridad, navegación, feedback, estados, accesibilidad y adecuación al contexto móvil.
Arquitectura	Separación de responsabilidades, dependencias, mantenibilidad y aplicación de MVVM / Clean Architecture.
Datos y persistencia	Coherencia entre fuentes locales, remotas y necesidades de persistencia.
Offline First	Comportamiento frente a conectividad intermitente o inexistente, cuando corresponda.
Implementación	Correcto uso de Kotlin, Android y Jetpack Compose.
Capacidades móviles	Uso pertinente de sensores, ubicación u otras APIs del dispositivo.
Repositorio	Evolución del trabajo, commits, participación e integración del equipo.
Defensa técnica	Capacidad individual y grupal para explicar decisiones, código y funcionamiento.

10. Cierre

El éxito del TPO no estará determinado por la cantidad de pantallas, calidad de la UI/UX, funcionalidades o tecnologías utilizadas.

Se valorará especialmente la capacidad de construir una solución **pequeña, coherente, defendible y funcional**, donde las decisiones de experiencia, arquitectura y tecnología puedan vincularse con necesidades reales del usuario.

Comprender el problema. Diseñar con intención. Implementar con criterio. Validar y poder explicarlo.